

Revisión de TourneyPro

Hallazgos y cambios propuestos — backend, autorización y frontend

■ Seguridad / autorización

1. updateTournament sin comprobar propietario

[backend/queries/queries.js:309-331](#)

A diferencia de updateTeam (que sí valida founder_id === req.user_id), cualquier usuario logueado puede editar (o desactivar) cualquier torneo. Además sobrescribe created_by con quien edita, perdiendo el autor original.

Cambios a hacer:

- Antes del UPDATE: SELECT created_by FROM tournaments WHERE tournament_id = ?
- Si no existe el torneo → 404.
- Si created_by !== req.user_id → 403 'No tienes permisos para editar este torneo'.
- En el UPDATE, dejar de pasar req.user_id como created_by; quitar ese parámetro de la query y de los binds — created_by no debe cambiar tras la creación.

2. updateGameData sin autorización + sport_id hardcodeado

[backend/queries/queries.js:409-453](#)

- Antes de actualizar, comprobar que req.user_id es el created_by del torneo: SELECT created_by, sport_id FROM tournaments WHERE tournament_id = ?. Si no coincide → 403.
- Usar el sport_id leído del torneo en vez de 'const sport_id = 1', y pasarlo al INSERT del siguiente partido.

3. tournamentInscription sin autorización ni límite de plazas

[backend/queries/queries.js:358-381](#)

- Comprobar que req.user_id pertenece al equipo: SELECT * FROM team_players WHERE team_id = ? AND user_id = ? (o que es el founder). Si no → 403.
- Antes del INSERT, contar equipos ya inscritos (SELECT COUNT(*) FROM tournament_teams WHERE tournament_id = ?) y compararlo con total_teams; si está lleno → 400 'El torneo ya tiene el máximo de equipos'.

4. register usa validación parcial

[backend/queries/queries.js:110](#)

Cambiar:

```
const result = validatePartialUserData(req.body);
```

por:

```
const result = validateUserData(req.body);
```

(el import ya trae validateUserData, solo falta usarlo aquí).

5. Logs con datos sensibles

[backend/querys/querys.js:47, 187-196, 211, 215](#)

- Línea 47: borrar `console.log(result)`; (expone el hash de la contraseña).
- Líneas 187-196, 211, 215: borrar los `console.log` de depuración del refresh token (comparaciones de hash, ids, filas afectadas). Se puede dejar un log de error genérico, pero sin volcar tokens/hashes.

6. Rate limiting en login

[backend/server.js](#)

- Instalar `express-rate-limit`.
- Crear un limiter (p. ej. 10 intentos / 15 min por IP) y aplicarlo solo a la ruta de login:

```
import rateLimit from 'express-rate-limit';
const loginLimiter = rateLimit({ windowMs: 15*60*1000, max: 10, standardHeaders: true,
legacyHeaders: false });
app.post('/api/login', loginLimiter, Querys.login);
```

7. CORS / helmet ausentes

[backend/server.js](#)

- Instalar `cors` y `helmet`.
- Antes de las rutas, añadir:

```
import cors from 'cors';
import helmet from 'helmet';
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));
```

Añadir `FRONTEND_URL` al `.env` (en dev sería `http://localhost:5173`, aunque con el proxy de Vite no es estrictamente necesario hasta que despliegues frontend/backend en orígenes distintos). Documentar esto para cuando se despliegue en producción.

■ Bugs funcionales (frontend)

8. `userContext.jsx`: crash al fallar el refresh

[src/context/userContext.jsx:1-23](#)

Cuando `/api/me` da 401 y el refresh falla, se llama a `navigate('/login')`, pero `navigate` no está importado ni definido ahí. Como el access token expira a los 15 minutos, en cuanto expire sin refresh token válido, la app revienta con `ReferenceError` en vez de redirigir a login.

Cambio a hacer:

```
import { useNavigate } from 'react-router-dom';
...
export function UserContextProvider({ children }) {
  const navigate = useNavigate();
  const [user, setUser] = useState(null);
```

Ojo: `UserContextProvider` debe renderizarse dentro de `<BrowserRouter>` para que `useNavigate` funcione — comprobar en `main.jsx/App.jsx` el orden de providers.

9. Menu.jsx: mismo patrón

[src/components/Menu.jsx:1-2, 38](#)

Añadir:

```
import { useNavigate } from "react-router-dom";
import { refreshToken } from '../services/refreshToken';
...
const navigate = useNavigate();
```

así handleLogout (líneas 38-41) deja de referenciar variables inexistentes.

10. Active-link de 'Mis Inscripciones' incorrecto

[src/components/Menu.jsx:94](#)

Cambiar:

```
activeLink === '/clasificacion'
```

por:

```
activeLink === '/inscripciones'
```

11. Typo 'Dashnoard'

[src/components/Menu.jsx:88](#)

Cambiar el texto 'Dashnoard' por 'Dashboard'.

■ A medio terminar

12. Activar o retirar Clasificación

[src/App.jsx:29](#), [src/components/Menu.jsx:93](#)

El componente Clasificacion.jsx parece completo y funcional (tabla de posiciones calculada desde getGames), pero tanto la ruta como el enlace del menú están comentados.

Si se da por buena:

- App.jsx:29 — descomentar la línea de la ruta /clasificacion.
- Menu.jsx:93 — descomentar el NavLink correspondiente.
- Revisar que el import de BarChart3 en Menu.jsx (ya presente) se use en ese enlace.

Si no se da por buena todavía, identificar qué le falta antes de exponerla.

13. Commits pendientes

No es un cambio de código, pero conviene commitear los archivos nuevos (CreateTeamForm.jsx, Calendario.jsx, Clasificacion.jsx, Inscripciones.jsx) junto con los arreglos de bugs en un commit separado de los cambios de seguridad, para poder revertir cada bloque de forma independiente si algo falla.

■ Calidad general

14. Validación Zod para equipos / inscripciones

[backend/schemas.js](#), [backend/queries/queries.js:455, 555](#)

En `schemas.js`, añadir:

```
const teamSchema = z.object({
  team_name: z.string().min(1),
  team_description: z.string().optional(),
  max_players: z.number().min(1),
  team_shield: z.string().optional(),
});
export function validateTeamData(data) { return teamSchema.safeParse(data); }
```

y usarlo en `createTeam/updateTeam` en vez de los `'if (!team_name || !max_players)'` manuales. Igual para `joinTeam/tournamentInscription`, validando que `teamId/tournamentId` sean números.

15. Mensaje de contraseña inconsistente

[backend/schemas.js:8](#)

Cambiar:

```
password: z.string().min(8, 'La contraseña debe tener al menos 6 caracteres')
```

por:

```
password: z.string().min(8, 'La contraseña debe tener al menos 8 caracteres')
```

16. Tests

No hay tests en el proyecto. Propuesta para más adelante: Vitest para frontend y Supertest (o similar) para los endpoints de `queries.js`. Es una tarea aparte y más grande — planificarla por separado.

Prioridad recomendada: punto 8 (crash real en producción diaria) y puntos 1-3 (autorización), que son los de mayor impacto.